

k6

Code-driven performance and load testing with metrics and thresholds.

What k6 is

k6 is a Grafana tool for load and performance testing. Tests are defined as code and can evolve with the application, be versioned with the project and run locally or in CI/CD. For QA, it helps turn workload assumptions and performance expectations into repeatable checks instead of relying only on manual observation.

What QA measures

Do not rely only on average response time. Review duration percentiles, failure rate, throughput, checks and application/infrastructure metrics. A test may have an acceptable average while a relevant portion of users experiences slow responses, so p90/p95/p99 and error behavior matter.

Load scenarios

Model the workload according to the objective: a small performance smoke, expected average load, peak traffic, stress beyond normal capacity or a longer soak/endurance scenario. Define virtual users or arrival rate, ramp-up, duration and realistic test data based on expected usage rather than arbitrary numbers.

Thresholds

Thresholds turn performance expectations into executable acceptance criteria. For example, define a maximum failure rate and a percentile target for request duration. A threshold should reflect a meaningful service expectation and make the test fail clearly when the agreed performance criterion is violated.

First practical flow

Install k6 from the official source, validate with `k6 version`, create a small script with `k6 new` or your editor and run it with `k6 run`. Start with a controlled endpoint and low load, inspect checks and metrics, then increase complexity only after the basic script and environment are understood.

CI/CD use

k6 can run in pipelines to detect performance regressions. Keep fast and deterministic checks in pull-request or deployment gates and schedule heavier load/stress/soak tests in appropriate environments and windows. Store results and compare trends so a pipeline does not become a source of noisy, unexplained failures.

Attention point

Do not generate meaningful load against production systems or third-party services without explicit authorization, capacity planning and monitoring. Performance results depend on environment, data, network and backend conditions; document those conditions before comparing executions.